Cryptography
Practical 8: RSA, DSA, and ECC

Martin Harrigan, ✉ martin.harrigan@setu.ie

In this practical, we will use Python's `pyca/cryptography` package to generate RSA, DSA, and ECC key pairs, and to use them for key encapsulation and for digital signatures. It follows the *Public-Key Cryptography*, *Digital Signatures*, and *Elliptic Curves* lectures. The documentation is at https://cryptography.io/en/latest/hazmat/primitives/asymmetric/, under RSA, DSA, Elliptic curve cryptography, and Ed25519 signing.

1 Key Pair Generation

First, we need to generate an RSA key pair:

```
from cryptography.hazmat.primitives.asymmetric import rsa

private_key = rsa.generate_private_key(
  public_exponent=65537,
  key_size=3072
)

public_key = private_key.public_key()
```

This generates a 3072 bit key pair, which provides 128 bit security. Note that the public key is derived from the private key. The private key, however, *cannot* be derived from the public key. If you wish to serialise the public and private keys and print them out, you can use the following code:

```
from cryptography.hazmat.primitives import serialization

private_key_str = private_key.private_bytes(
  encoding=serialization.Encoding.PEM,
  format=serialization.PrivateFormat.PKCS8,
  encryption_algorithm=serialization.NoEncryption()
)

# We can print out the private key.
print(private_key_str.decode("utf-8"))

public_key_str = public_key.public_bytes(
  encoding=serialization.Encoding.PEM,
  format=serialization.PublicFormat.SubjectPublicKeyInfo
)

# We can print out the public key.
print(public_key_str.decode("utf-8"))
```

A DSA key pair is generated in the same way, with `dsa.generate_private_key(key_size=3072)`. An ECC key pair needs an elliptic curve rather than a key size: `ec.generate_private_key(ec.SECP256R1())`. Ed25519 has a class of its own: `ed25519.Ed25519PrivateKey.generate()`. Table 1 gives the parameters for each security level.

1.1 Your Task

Generate a key pair of each kind in the 128 bit row of Table 1, and print each public key with the code above. Which public key is the longest, and which the shortest? Now generate a 7680 bit RSA key pair and a P-384 key pair, for 192 bit security. How much longer is each public key?

Security (bit)	RSA (bit)	DSA (bit)	NIST Curves	Edwards Curves
128	3072	3072	P-256	Ed25519
192	7680		P-384	
256	15,360		P-521	

Table 1: Key sizes and curves for each security level. The `pyca/cryptography` package generates DSA keys of at most 4096 bit, so DSA stops at 128 bit. A 15,360 bit RSA key takes minutes to generate.

2 RSA and OAEP

RSA encrypts a number $m < n$ as $c = m^e \bmod n$. Used directly like this, it is insecure. It is deterministic: the same plaintext always gives the same ciphertext, so an attacker who guesses the plaintext (`yes` or `no`, say) can encrypt the guess under the public key and compare. It is also malleable: an attacker can change a ciphertext into the ciphertext of a related plaintext without knowing either plaintext.

Optimal asymmetric encryption padding (OAEP) prepares the message before RSA is applied. It mixes the message with a fresh random seed, so encrypting the same message twice gives two different ciphertexts. It also adds a fixed pattern that Bob checks after decrypting, so a ciphertext that has been changed is rejected rather than decrypted to something else.

The mixing needs something like a hash function whose output is as long as the message block, rather than a fixed 32 B. A mask generation function (MGF) provides this: it stretches a short input into an output of any length that looks random. MGF1, the one in use everywhere, does it by hashing the input again and again with a counter and joining the results. This is what `padding.MGF1(hashes.SHA256())` means in the sample code.

2.1 Your Task

Section A provides sample Python code for generating an RSA key pair, encrypting a small random plaintext under the public key with OAEP, and decrypting the ciphertext with the private key.

1. In Section A, change `os.urandom(50)` to `os.urandom(5000)`. The code should throw an exception with the message "ValueError: Encryption failed."
2. Encrypt the same plaintext twice and compare the two ciphertexts. Why do they differ? How would an attacker who guesses the plaintext benefit if they did not?
3. Change a single byte of the ciphertext before decrypting it. What happens? The exception says only that decryption failed. Why might it be unwise to say more?

3 Key Encapsulation Mechanisms

RSA is not used to encrypt messages directly. Instead, it is used as a key encapsulation mechanism (KEM), which has three operations:

1. `keygen`: Bob generates a key pair and shares the public key.
2. `encapsulate`: Alice uses randomness and Bob's public key to produce a secret and an *encapsulation* (a ciphertext), which she sends to Bob.
3. `decapsulate`: Bob uses his private key to recover the same shared secret from the encapsulation.

With RSA, Alice encapsulates by generating 32 B of random data and encrypting it under Bob's public key with OAEP. Bob decapsulates by decrypting. The secret then serves as the key of a symmetric cipher that encrypts the message. A secret is usually passed through a key derivation function first; this one is already 32 B of random data, so it can be used as an AES key directly.

Section B provides sample Python code that does this in two layers:

- `encapsulate()` generates a 32 B secret and encrypts it with RSA-OAEP; `decapsulate()` decrypts it.
- `encrypt_message()` uses the secret as a 256 bit AES key and encrypts the message with AES-GCM. `decrypt_message()` reverses both steps.

AES-GCM is an *authenticated* cipher: decryption fails if the ciphertext has been changed. The *Authenticated Encryption* lecture covers it in detail; for now, note that it encrypts a message of any length in one call.

1. Run the code. Compare the total size of what Alice sends (the encapsulation, the nonce, and the ciphertext) with the size of the message. Where does each extra byte come from?
2. Change the message to 10 MiB (`os.urandom(10 * 1024 * 1024)`). How many RSA operations does Alice perform now?
3. Change a single byte of the encapsulation before Bob decrypts. What happens, and in which step?
4. Alice wants to send the same large message to three people. What must she do three times, and what only once?

3.1 Elliptic Curve Diffie–Hellman as a KEM

Diffie–Hellman with a fresh (*ephemeral*) key pair on Alice's side is also a KEM. With ECDH, Alice encapsulates by generating an ephemeral key pair on the same curve as Bob's, performing the key exchange (`ephemeral.exchange(ec.ECDH(), bob_public)`) with Bob's public key, and converting her ephemeral public key to bytes, which are the encapsulation:

```
encapsulation = ephemeral.public_key().public_bytes(
  serialization.Encoding.X962,
  serialization.PublicFormat.UncompressedPoint
)
```

Bob decapsulates by loading the ephemeral public key from those bytes and performing the key exchange with it. The method that loads it is `ec.EllipticCurvePublicKey.from_encoded_point(curve, data)`.

3.2 Your Task

Write `encapsulate()` and `decapsulate()` functions for ECDH on P-256, with the same signatures as those in Section B, and check that Alice and Bob agree on the secret. Compare the length of the encapsulation with that of a 3072 bit RSA encapsulation, which gives the same security.

4 Digital Signatures

Each of the following signature schemes uses the *hash-then-sign* technique: the message is hashed, and the hash value is signed.

RSA Section C provides sample Python code for generating an RSA key pair, signing a random message with the private key, and verifying the signature with the public key. It uses probabilistic signature scheme (PSS) padding, which does for signatures what OAEP does for encryption: it mixes the hash value with a random *salt*, using MGF1, before RSA is applied. Signing the same message twice therefore gives two different signatures.

DSA The Digital Signature Algorithm (DSA) was developed by the National Institute of Standards and Technology (NIST) in conjunction with the National Security Agency (NSA) and published in 1991. It is designed for digital signatures only: unlike RSA, it does not support encryption and decryption. Since 2023, FIPS 186-5 allows DSA only for verifying existing signatures, not for making new ones, but its successor works the same way in a different group. Section D provides sample Python code that signs and verifies with a 3072 bit DSA key pair.

ECDSA The elliptic curve digital signature algorithm (ECDSA) is DSA on an elliptic curve, and gives the same security with much smaller keys. Section E provides sample Python code that signs and verifies with P-256. Choose a hash function as strong as the curve: SHA-256 with P-256, SHA-384 with P-384, and SHA-512 with P-521 (see Table 2). Both DSA and ECDSA need a fresh random number for every signature; reusing one for two signatures reveals the private key.

EdDSA Ed25519 is based on Curve25519 and uses the Edwards-curve digital signature algorithm (EdDSA) defined in RFC 8032. An Edwards curve is an elliptic curve written in a different form, whose addition formulas are faster and easier to implement in constant time than those of the NIST curves. Unlike ECDSA, Ed25519 uses a separate module and signing API: you pass the message directly to `private_key.sign(message)` and do not choose a hash function. It also derives the per-signature number from the private key and the message, rather than from a random number generator, so a weak generator cannot reveal the key.

Curve	Class in `pyca/cryptography`	Security	Hash
NIST P-256	`ec.SECP256R1`	128 bit	SHA-256
NIST P-384	`ec.SECP384R1`	192 bit	SHA-384
NIST P-521	`ec.SECP521R1`	256 bit	SHA-512
SECP256K1	`ec.SECP256K1`	128 bit	SHA-256
Ed25519	`ed25519.Ed25519PrivateKey`	128 bit	built in

Table 2: Elliptic curves in the `pyca/cryptography` package, with their security levels and the hash function to sign with. SECP256K1 is used by Bitcoin and Ethereum.

4.1 Your Task

Generate one random 1 KiB message, `os.urandom(1024)`. Sign it twice with each of 3072 bit RSA-PSS, 3072 bit DSA, ECDSA on P-256, and Ed25519, and verify each signature.

1. Which schemes give the same signature twice, and which give two different ones? Why?
2. How long is each signature in bytes? Why do the DSA and ECDSA signatures not always have the same length?
3. Change a single byte of the message and verify the signature again. What happens?

Appendices

A Python Code That Uses RSA To Encrypt Small Messages

```
import os

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding, rsa

# A 3072-bit key gives about 128-bit security.
private_key = rsa.generate_private_key(public_exponent=65537, key_size=3072)
public_key = private_key.public_key()

# OAEP padding, with SHA-256 as the hash and inside MGF1.
oaep = padding.OAEP(mgf=padding.MGF1(hashes.SHA256()), algorithm=hashes.SHA256(), label=None)

plaintext = os.urandom(50)

# We can encrypt a small plaintext with the public key.
ciphertext = public_key.encrypt(plaintext, oaep)

# Only the private key can decrypt it.
recovered = private_key.decrypt(ciphertext, oaep)

print("Plaintext: " + plaintext.hex())
print()
print("Ciphertext: " + ciphertext.hex())
print()
print("Recovered: " + recovered.hex())
```

Listing 1: rsa_encrypt_decrypt.py

B Python Code That Uses RSA as a KEM

```
"""RSA as a KEM protects a fresh secret; AES-GCM protects the whole message."""

import os

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding, rsa
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

OAEP = padding.OAEP(mgf=padding.MGF1(hashes.SHA256()), algorithm=hashes.SHA256(), label=None)


# The key encapsulation mechanism (KEM): Alice encapsulates a fresh random
# secret under Bob's public key, and only Bob can decapsulate it.


def encapsulate(public_key):
    # 32 random bytes, so the secret can serve directly as an AES-256 key.
    secret = os.urandom(32)
    return secret, public_key.encrypt(secret, OAEP)


def decapsulate(private_key, encapsulation):
    return private_key.decrypt(encapsulation, OAEP)


# The data encapsulation: the secret encrypts the message with AES-GCM.
```

```
def encrypt_message(public_key, message):
    secret, encapsulation = encapsulate(public_key)
    nonce = os.urandom(12)
    ciphertext = AESGCM(secret).encrypt(nonce, message, None)
    return encapsulation, nonce, ciphertext


def decrypt_message(private_key, encapsulation, nonce, ciphertext):
    secret = decapsulate(private_key, encapsulation)
    return AESGCM(secret).decrypt(nonce, ciphertext, None)


private_key = rsa.generate_private_key(public_exponent=65537, key_size=3072)
message = os.urandom(10 * 1024)
encapsulation, nonce, ciphertext = encrypt_message(private_key.public_key(), message)
assert decrypt_message(private_key, encapsulation, nonce, ciphertext) == message
print("Message recovered.")

print()
print("Message bytes:", len(message))
print("Encapsulation bytes:", len(encapsulation))
print("Nonce bytes:", len(nonce))
print("Ciphertext bytes, including the 16-byte tag:", len(ciphertext))
```

Listing 2: rsa_hybrid_encrypt_decrypt.py

C Python Code That Uses RSA To Sign Messages

```
import os

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding, rsa

# RSA signing with PSS padding (Probabilistic Signature Scheme).
# A 3072-bit key gives about 128-bit security.
private_key = rsa.generate_private_key(public_exponent=65537, key_size=3072)
public_key = private_key.public_key()

# PSS padding, with SHA-256 inside MGF1 and a salt the length of the hash.
pss = padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.DIGEST_LENGTH)

message = os.urandom(1024)

# We can sign the message using "hash-then-sign".
signature = private_key.sign(message, pss, hashes.SHA256())

# We can verify the signature. If the signature is invalid it will
# raise an Exception.
public_key.verify(signature, message, pss, hashes.SHA256())

print()
print("Message: " + message.hex())
print()
print("Signature: " + signature.hex())
```

Listing 3: rsa_sign_verify.py

D Python Code That Uses DSA To Sign Messages

```
import os

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import dsa

# DSA signs but cannot encrypt. FIPS 186-5 allows it only for verifying
# existing signatures. A 3072-bit key gives about 128-bit security.
private_key = dsa.generate_private_key(key_size=3072)
public_key = private_key.public_key()

message = os.urandom(1024)

# We can sign the message using "hash-then-sign".
signature = private_key.sign(message, hashes.SHA256())

# We can verify the signature. If the signature is invalid it will
# raise an Exception.
public_key.verify(signature, message, hashes.SHA256())

print()
print("Message: " + message.hex())
print()
print("Signature: " + signature.hex())
```

Listing 4: dsa_sign_verify.py

E Python Code That Uses ECDSA To Sign Messages

```
import os

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import ec

# ECDSA with P-256 and SHA-256, about 128-bit classical security.
private_key = ec.generate_private_key(ec.SECP256R1())
public_key = private_key.public_key()

message = os.urandom(1024)

# We can sign the message using "hash-then-sign".
signature = private_key.sign(message, ec.ECDSA(hashes.SHA256()))

# We can verify the signature. If the signature is invalid it will
# raise an Exception.
public_key.verify(signature, message, ec.ECDSA(hashes.SHA256()))

print()
print("Message: " + message.hex())
print()
print("Signature: " + signature.hex())
```

Listing 5: ecc_sign_verify.py